

Module 1

The History and Evolution of Java: The Byte code, Features of Java An overview of Java: Object-Oriented Programming, Structure of a Java program, Data Types and Variables, Type conversion and casting, Arrays

Classes: Fundamentals, Declaring Objects, Assigning Object Reference Variables, Methods, Constructors, this Keyword, Garbage Collection, Stack application

Methods and Classes: Overloading Methods, Using Objects as Parameters, Argument Passing, Returning Objects, Access Control, static, final, Command-Line Arguments

The History and Evolution of Java

Java is related to C++, which is a direct descendant of C. Much of the character of Java is inherited from these two languages. From C, Java derives its syntax. Many of Java's object oriented features were influenced by C++.

C Language :

- Fundamentally changed the way programming was approached.
- Structured, efficient, high-level language that could replace assembly code.
- Invented and first implemented by Dennis Ritchie on UNIX operating system

C++

- The increasing complexity of programs has driven the need for better ways to manage that complexity.
- With structured programming methods, once a project reaches a certain size, its complexity exceeds.
- To solve this problem, a new way to program was invented, called *object-oriented programming (OOP)*.
- OOP is a programming methodology that helps organize complex programs through the use of inheritance, encapsulation, and polymorphism.
- C++ was invented by Bjarne Stroustrup in 1979.
- C++ extends C by adding object-oriented features.
- It includes all of C's features, attributes, and benefits.

Java

- Java was conceived by James Gosling, Patrick Naughton, Chris Warth, Ed Frank, and Mike Sheridan at Sun Microsystems.
- The primary motivation was the need for a platform-independent (that is, architecture-neutral) language that could be used to create software to be embedded in various consumer electronic devices, such as microwave ovens and remote controls.
- The trouble with C and C++ (and most other languages) is that they are designed to be compiled for a specific target
- The problem is that compilers are expensive and time-consuming to create.
- While java was being developed the world wide web was emerging in parallel and web demanded portable programs.
- Java derives much of its character from C and C++.

How Java changed the internet

- Java had a profound effect on the Internet.
- Java simplified web programming and also made networked programs available through servlets and applets.

Applets

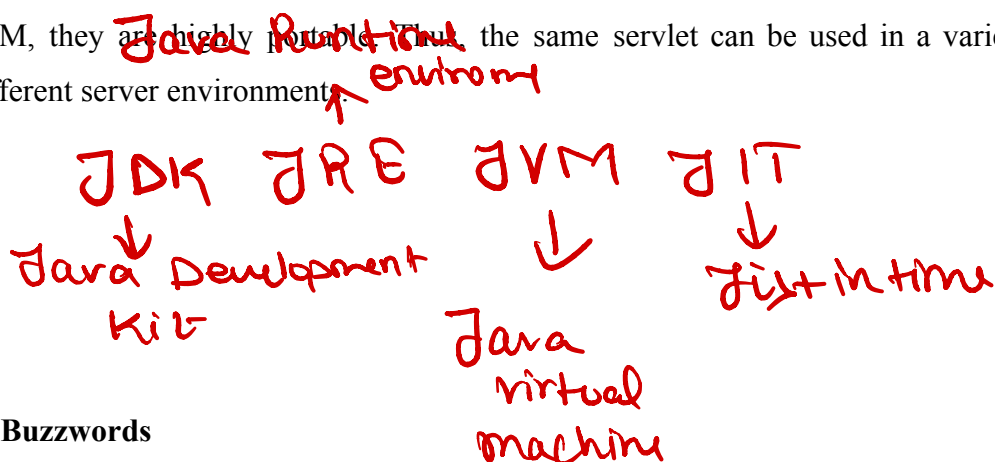
- An *applet* is a special kind of Java program that is designed to be transmitted over the Internet and automatically executed by a Java-compatible web browser.
- An applet is downloaded on demand, without further interaction with the user.
- Applets are intended to be small programs. They are typically used to display data provided by the server, handle user input, or provide simple functions, such as a loan calculator, that execute locally.
- Applet is a dynamic, self-executing program.
- Such a program that is downloaded automatically and run should be prevented from doing any harm and also be able to run in a variety of different environments and under different operating systems.
- Security is provided by confining an applet to the Java execution environment and not allowing it access to other parts of the computer.

The ByteCode

- The output of a Java compiler is not executable code, rather, it is bytecode.
- *Bytecode* is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the *Java Virtual Machine (JVM)*.
- Translating a Java program into bytecode makes it much easier to run a program in a wide variety of environments because only the JVM needs to be implemented for each platform.
- Bytecode has been highly optimized, the use of bytecode enables the JVM to execute programs much faster than one might expect.
- When a JIT compiler is part of the JVM, selected portions of bytecode are compiled into executable code in real time, on a piece-by-piece, demand basis.
- JIT compiler compiles code as it is needed, during execution.

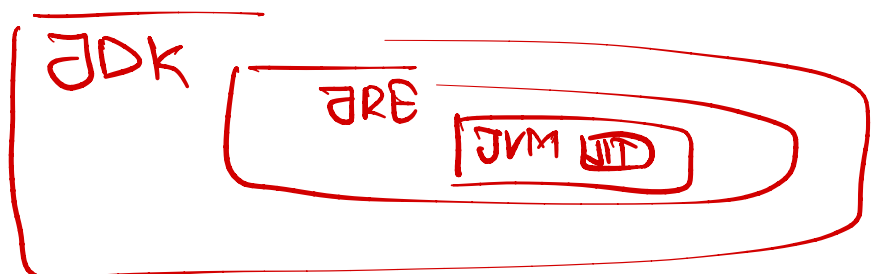
✓ Servlets

- A servlet is a small program that executes on the server.
- Just as applets dynamically extend the functionality of a web browser, servlets dynamically extend the functionality of a web server.
- Servlets are used to create dynamically generated content that is then served to the client.
- The servlet offers several advantages, including increased performance.
- Servlets (like all Java programs) are compiled into bytecode and executed by the JVM, they are highly portable and the same servlet can be used in a variety of different server environments.



The Java Buzzwords

1. Simple



- a. Java was designed to be easy for the professional programmer to learn and use effectively.
- b. With knowledge of C and Object oriented principles Java is a simple language to be learnt.

2. Secure

- a. Java confines the running of code to JVM only and does not let it to access the other parts of the computer.

3. Portable

- a. Java is known as a portable language because Java code can execute on all major platforms.
- b. Bytecode, A highly optimized set of instructions makes java a portable programming language.
- c. Java bytecode on any hardware that has a compliant JVM.

4. Object oriented

- a. The object oriented principles are Encapsulation, Abstraction, Inheritance and polymorphism.
- b. Encapsulation refers to binding together of code and data.
- c. Abstraction refers to hiding of complex features.
- d. Inheritance is a property through which one class can obtain the features of the other.
- e. Polymorphism means “One entity multiple forms”.
- f. The objected oriented model allows the user to incorporate all the features mentioned above into the code.
- g. Everything in Java is considered an object and hence programs are modular.

5. Robust

(Try catch)

- a. Java restricts the programmer in a few key areas thus helping them identify mistakes early in the program development.
- b. Compile time checks helps identifying programming errors.
- c. Run time errors can be checked through Exceptions handling mechanism.
- d. Memory management is a tedious task in C and C++. The programmer needs to manually allocate and free the dynamic memory.
- e. Java's memory allocation is simple and deallocation of memory is automatic through garbage collection.

- f. Exceptional conditions like file not found, array index out of bound, divide by zero can be easily handled through Exception handling mechanisms.

6. Multithreaded

- a. Java allows multithreading i.e. it allows the user to write programs that can do many things simultaneously.
- b. The Java run time provides solutions for multiprocess synchronizations that helps the user to construct interactive programs.

7. Architecture neutral

- a. “ Write once run anywhere , anytime forever” was the goal with which Java was created and the goal is accomplished to a great extent.
- b. Operating system upgrades , processor upgrades and changes in the core system resources do not affect the bytecode runs.

8. Interpreted and high performance

- a. Java enables creation of cross platform programs by compiling into an intermediate representation called the bytecode.
- b. This code can be run on any machine with a JVM.
- c. Java bytecode was carefully designed so that it would be easy to translate directly into native machine code by using the Just in time compiler.

9. Distributed

- a. Java can handle the TCP/IP protocols.
- b. Java also supports Remote Method Invocation which enables a program to invoke methods across network.

10. Dynamic

- a. Java programs carry substantial amounts of run time information that is used to verify and resolve access to objects at run time.
- b. This helps in linking the code dynamically in a safe manner thus making the language robust.

An Overview of Java

Object-Oriented Programming

- Object-oriented programming (OOP) is at the core of Java.

- All computer programs consist of two elements: code and data.
- To manage increasing complexity *object-oriented programming* was introduced.
- Object-oriented programming organizes a program around its data (that is, objects) and a set of well-defined interfaces to that data.
- An object-oriented program can be characterized as *data controlling access to code*.

OOP Principles

1. Abstraction :

- An essential element of object-oriented programming is *abstraction*.
- Data abstraction is the process of hiding certain details and showing only essential information to the user.
- It helps to reduce programming complexity and effort.
- For example, people do not think of a car as a set of tens of thousands of individual parts. They think of it as a well-defined object with its own unique behaviour.
- They can ignore the details of how the engine, transmission, and braking systems work. Instead, they are free to utilize the object as a whole.
- In Java abstraction is achieved through abstract classes and interface.

2. Encapsulation

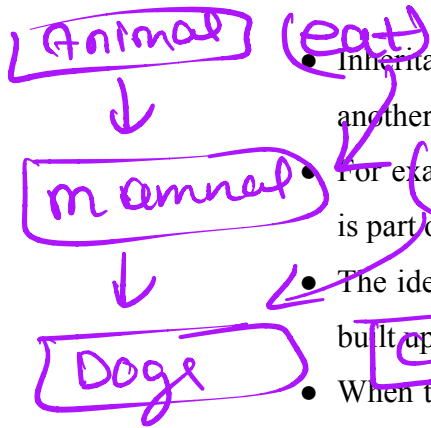
(Private, Public, Protected)

- Encapsulation is the mechanism that binds together code and the data it manipulates keeps data and code safe from outside interference and misuse.
- Encapsulation can be thought of as a protective wrapper that prevents the code and data from being arbitrarily accessed by other code defined outside the wrapper.
- It is also known as “Data hiding”.
- In Java, the basis of encapsulation is the class. Each method or variable in a class may be marked private or public.
- The *public* interface of a class represents everything that external users of the class may know.

- The *private* methods and data can only be accessed by code that is a member of the class.

3. Inheritance

(extends)



- Inheritance is the process by which one object acquires the properties of another object.
- For example a Golden Retriever is part of the classification *dog*, which in turn is part of the *mammal* class, which is under the larger class *animal*.
- The idea behind inheritance in Java is that one can create new classes that are built upon existing classes.
- When the user inherits from an existing class, he can reuse methods and fields of the parent class. Also, one can add new methods and fields in the current class.
- Inheritance facilitates Reusability and hence is an important concept of OOPs.

4. Polymorphism

(method Overload / constructor Overloading)

- The word polymorphism means having many forms.. Polymorphism allows us to perform a single action in different ways.
- For Example: Consider a stack (which is a last-in, first-out list). There might be a need for 3 types of stack, one for integer values one for double and one for string. The algorithm that implements each stack is the same, even though the data being stored differs. In a non-object-oriented language, one had to create three different sets of stack routines, with each set using different names. However, because of polymorphism, in Java one can specify a general set of stack routines that all share the same names.
- Polymorphism in java can be implemented by overloading and overriding.

A Simple Java program

```
/* This is a simple Java program. */
```

```
class Example {
```

} → int a, b, c
 } → state
 → method {

3 3 for }

```
public static void main(String args[]) {

    System.out.println("This is a simple Java program.");

}
```

To compile

C:\>javac Example.java

The **javac** compiler creates a file called **Example.class** that contains the bytecode version of the program

To run the code call the java interpreter: C:\>java Example

Output : This is a simple Java program.

`/* This is a simple Java program*/` : This is a *comment*. Like most other programming languages, Java lets one to enter a remark into a program's source file. The contents of a comment are ignored by the compiler.

`class Example {` : This line uses the keyword **class** to declare that a new class is being defined. **Example** is an *identifier* that is the name of the class. The entire class definition, including all of its members, will be between the curly braces `{ }`.

`public static void main(String args[ ])` : This line begins the **main()** method. The **public** keyword is an *access modifier*, When a class member is preceded by **public**, then that member may be accessed by code outside the class in which it is declared. The keyword **static** allows **main()** to be called without having to instantiate a particular instance of the class. The keyword **void** simply tells the compiler that **main()** does not return a value. **String args[]** declares a parameter named **args**, which is an array of instances of the class **String**.

Data types and variables.

Java defines eight primitive types of data: byte, short, int, long, char, float, double, and

int a = 5 → memory location 16F
 int b = 8a
 b → 16F → dereference
 C = *a.

boolean.

These can be put in four groups:

✓ **Integers:** This group includes byte, short, int, and long, which are for whole-valued signed numbers.

✓ **Floating-point numbers:** This group includes float and double, which represent numbers with fractional precision.

✓ **Characters:** This group includes char, which represents symbols in a character set, like letters and numbers.

✓ **Boolean:** This group includes boolean, which is a special type for representing true/false values.

Type Conversion and Casting

It is possible to assign a value of one type to a variable of another type. If the two types are compatible, then Java will perform the conversion automatically. For example, it is always possible to assign an **int** value to a **long** variable. However, not all types are compatible, and thus, not all type conversions are implicitly allowed. For instance, there is no automatic conversion defined from **double** to **byte**. But it is still possible to obtain a conversion between incompatible types. To do so, a cast must be used, which performs an explicit conversion between incompatible types.

Java's Automatic Conversions

When one type of data is assigned to another type of variable, an *automatic type conversion* will take place if the following two conditions are met:

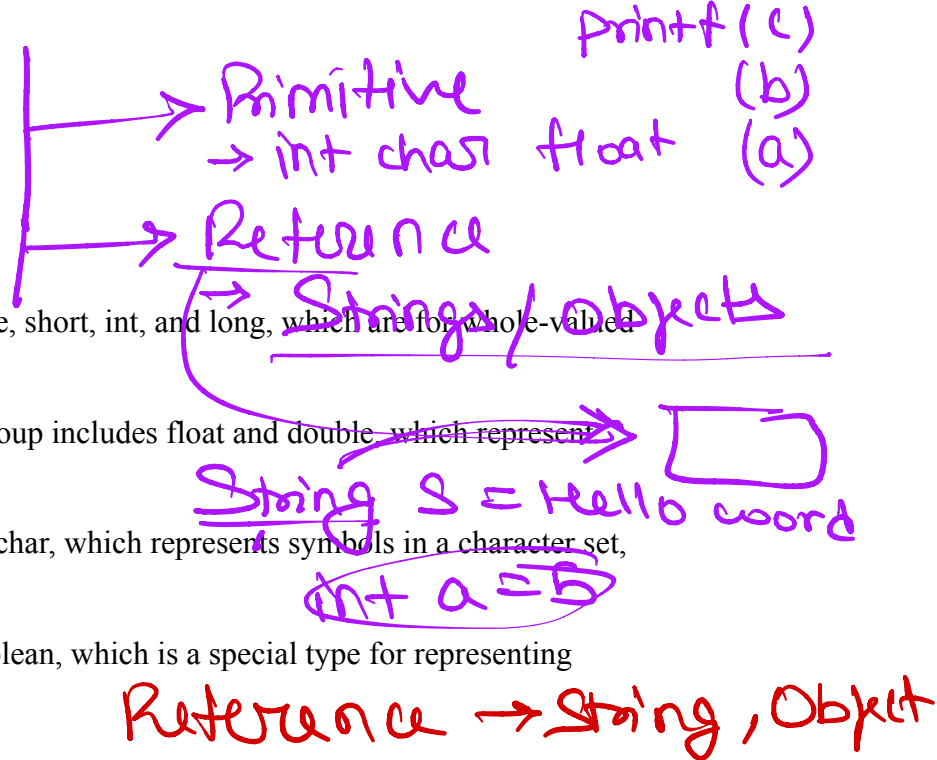
- The two types are compatible.
- The destination type is larger than the source type.

When these two conditions are met, a *widening conversion* takes place. For example, the **int** type is always large enough to hold all valid **byte** values, so no explicit cast statement is required.

For widening conversions, the numeric types, including integer and floating-point types, are compatible with each other. However, there are no automatic conversions from the numeric types to **char** or **boolean**. Also, **char** and **boolean** are not compatible with each other.

Casting Incompatible Types

Although the automatic type conversions are helpful, they will not fulfill all needs. For example, if there is a need to assign an **int** value to a **byte** variable, this conversion will not be performed automatically, because a **byte** is smaller than an **int**. This kind of conversion is



→ int a = 5
float b = 5.5

c = a + b
↑ float automatic

called a *narrowing conversion*, since the value is explicitly made narrower, so that it will fit into the target type. To create a conversion between two incompatible types, a cast must be used. A *cast* is simply an explicit type conversion. It has this general form:

(target-type) value

Here, *target-type* specifies the desired type to convert the specified value to. For example, the following fragment casts an **int** to a **byte**. If the integer's value is larger than the range of a **byte**, it will be reduced modulo (the remainder of an integer division by the) **byte**'s range.

```
int a;  
byte b;  
// ...  
b = (byte) a;
```

A different type of conversion will occur when a floating-point value is assigned to an integer type: *truncation*. Integers do not have fractional components. Thus, when a floating-point value is assigned to an integer type, the fractional component is lost.

```
class Conversion {  
public static void main (String args[]) {  
    byte b;  
    int i = 257;  
    double d = 323.142;  
    System.out.println("\nConversion of int to byte.");  
    b = (byte) i;  
  
    System.out.println("i and b " + i + " " + b);  
    System.out.println("\nConversion of double to int.");  
    i = (int) d;  
  
    System.out.println("d and i " + d + " " + i);  
    System.out.println("\nConversion of double to byte.");  
    b = (byte) d;  
    System.out.println("d and b " + d + " " + b);  
    }  
}
```

This program generates the following output:

```
Conversion of int to byte.  
i and b 257 1  
Conversion of double to int.  
d and i 323.142 323  
Conversion of double to byte.  
d and b 323.142 67
```

When the value 257 is cast into a byte variable, the result is the remainder of the division of 257 by 256 (the range of a byte), which is 1 in this case. When the d is converted to an int, its

fractional component is lost. When d is converted to a byte, its fractional component is lost, and the value is reduced modulo 256, which in this case is 67.

Arrays

An *array* is a group of like-typed variables that are referred to by a common name. A specific element in an array is accessed by its index. The general form of a one-dimensional array declaration is *type var-name[]*;

```
int month_days[ ];
```

Although this declaration establishes the fact that **month_days** is an array variable, no array actually exists. In fact, the value of **month_days** is set to **null**, which represents an array with no value. To link **month_days** with an actual, physical array of integers, user must allocate one using **new** and assign it to **month_days**. **new** is a special operator that allocates memory.

```
month_days = new int[12];
```

A simple program on array

```
class Array {  
public static void main(String args[]) {  
int a[] = new int[5];  
for(int i=0; i<4; i++){  
a[i] = i;  
System.out.println(a[i]);  
}  
}
```

Declaring a 2D array:

```
int twoD[ ][ ] = new int[4][5];
```

This allocates a 4 by 5 array and assigns it to twoD.

~~int~~ a[] = { 1, 2, 3, 4, 5 }

1D
2D

a[][]

~~String~~ a[] = { "Hi", "Hello" }

Char

a[1][4]

0 → { 1, 2, 3, 4, 5 }
1 → { 1, 2, 3, 4, 5 }
↑ ↑ ↑ ↑ ↑
0 1 2 3 4

Introducing classes:

Classes are the core of Java. A class defines a new data type. Once defined, this new type can be used to create objects of that type. Thus, a class is a *template* for an object, and an object is an *instance* of a class.

General form of a class

When a class is defined, its exact form and nature are declared. The data that the class contains and the code that operates on the data have to be specified.

A class is declared by use of the **class** keyword.

```
class classname {  
  
    type instance-variable1;  
    type instance-variable2;  
    // ...  
    type instance-variableN;  
  
    type methodname1(parameter-list) {  
        // body of method  
    }  
    type methodname2(parameter-list) {  
        // body of method  
    }  
    // ...  
    type methodnameN(parameter-list) {  
        // body of method  
    }  
}
```

The data, or variables, defined within a **class** are called *instance variables*. The code is contained within *methods*. Collectively, the methods and variables defined within a class are called *members* of the class. Variables defined within a class are called instance variables.

A simple class

```
class Box {  
    double width;  
    double height;  
    double depth;  
    void volume() {  
        System.out.print("Volume is ");  
        System.out.println(width * height * depth);  
    }  
}  
  
class BoxDemo {  
    public static void main(String args[]) {  
        Box mybox1 = new Box();  
        mybox1.width = 10;  
    }  
}
```

```

        mybox1.height = 20;
        mybox1.depth = 15;

        mybox1.volume();
    }
}

```

Creating instances of a class

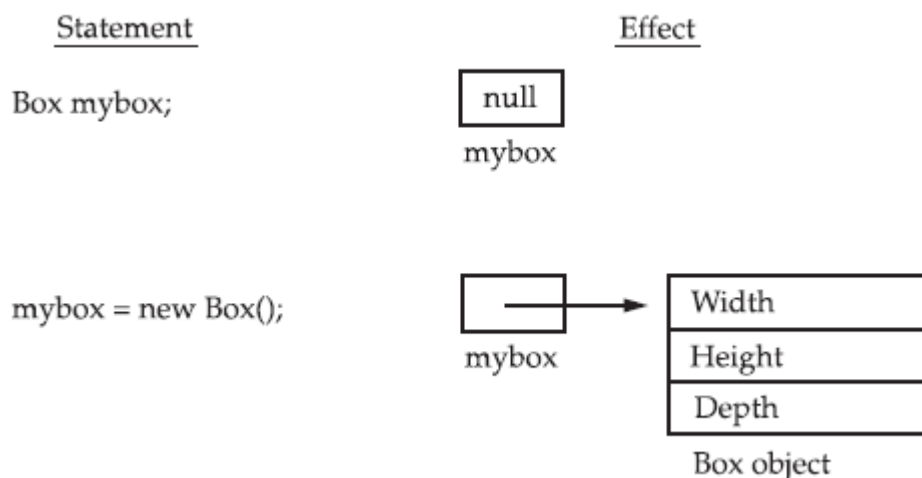
When a class is created, a new data type is created. This new data type can be used to declare objects of that type. Obtaining objects of a class is a 2 step process.

1. Declare a variable of the class type : This variable does not define an object but is a variable that can *refer* to an object.
2. Acquire an actual, physical copy of the object and assign it to that variable : This can be done using the **new** operator. The **new** operator dynamically allocates memory for an object and returns a reference to it.

```
Box mybox = new Box();
```

The above statement combines the two steps. It can be rewritten like below to show each step more clearly:

```
Box mybox; // declare reference to object
mybox = new Box(); // allocate a Box object
```



Constructors

A *constructor* initializes an object immediately upon creation. It has the same name as the class in which it resides and is syntactically similar to a method. Once defined, the constructor is automatically called immediately after the object is created, before the **new** operator completes. Constructors do not have a return type, not even **void**.

```

class Box {
    double width;
    double height;
    double depth;
    // This is the constructor for Box.
}

```

```

Box() {
    System.out.println("Constructing Box");
    width = 10;
    height = 10;
    depth = 10;
}
// compute and return volume
double volume() {
    return width * height * depth;
}
}
class BoxDemo{
    public static void main(String args[]) {

        Box mybox1 = new Box();

        double vol;

        vol = mybox1.volume();
        System.out.println("Volume is " + vol);
    }
}

```

Output

Constructing Box

Volume is 1000.0

The constructor in the above program contains no parameters. However a constructor can also have parameters and those are termed as parameterized constructors.

```

class Box {
    double width;
    double height;
    double depth;
// This is a parameterized constructor for Box.
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    double volume() {
        return width * height * depth;
    }
}

class BoxDemo {
    public static void main(String args[]) {

        Box mybox1 = new Box(10, 20, 15);
        double vol;
    }
}

```

```

        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

    }
}

```

Output

Volume is 3000.0

“this” keyword

“**this**” can be used inside any method to refer to the current object. It is illegal in Java to declare two local variables with the same name inside the same or enclosing scopes. One can have local variables, including formal parameters to methods, which overlap with the names of the class’ instance variables. Because **this** lets user to refer directly to the object, one can use it to resolve any namespace collisions that might occur between instance variables and local variables.

```

Box(double width, double height, double depth) {
    this.width = width;
    this.height = height;
    this.depth = depth;
}

```

Garbage Collection

Objects are dynamically allocated by using the **new** operator. Objects should be destroyed and their memory must be released for later reallocation. In C++, dynamically allocated objects must be manually released by use of a **delete** operator. Java handles deallocation automatically. The technique that accomplishes this is called *garbage collection*. When no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed. Different Java run-time implementations will take varying approaches to garbage collection.

The finalize() method

An object will need to perform some action before it is destroyed. If an object is holding some non-Java resource such as a file handle, it has to be freed before an object is destroyed. To handle such situations, Java provides a mechanism called *finalization*. By using finalization, one can define specific actions that will occur when an object is just about to be reclaimed by the garbage collector.

The finalize() method has this general form:

```

protected void finalize( )
{

```

```
// finalization code here
}
```

Stack Class – An example

```
class Stack {
    int stck[] = new int[10];
    int tos;

    // Initialize top-of-stack
    Stack() {
        tos = -1;
    }

    // Push an item onto the stack
    void push(int item) {
        if(tos==9)
            System.out.println("Stack is full.");
        else
            stck[++tos] = item;
    }

    // Pop an item from the stack
    int pop() {
        if(tos < 0) {
            System.out.println("Stack underflow.");
            return 0;
        }
        else
            return stck[tos--];
    }
}
```

The Stack class defines two data items and three methods. The stack of integers is held by the array `stck`. This array is indexed by the variable `tos`, which always contains the index of the top of the stack. The `Stack()` constructor initializes `tos` to `-1`, which indicates an empty stack. The method `push()` puts an item on the stack. To retrieve an item, `pop()` is called.

The class `TestStack`, demonstrates the Stack class. It creates two integer stacks, pushes some values onto each, and then pops them off.

```
class TestStack {
    public static void main(String args[]) {
        Stack mystack1 = new Stack();
        Stack mystack2 = new Stack();

        // push some numbers onto the stack
        for(int i=0; i<10; i++) mystack1.push(i);
        for(int i=10; i<20; i++) mystack2.push(i);
    }
}
```



```

// pop those numbers off the stack
System.out.println("Stack in mystack1:");
for(int i=0; i<10; i++)
System.out.println(mystack1.pop());

System.out.println("Stack in mystack2:");
for(int i=0; i<10; i++)
System.out.println(mystack2.pop());
}
}
This program generates the following output:
Stack in mystack1:
9
8
7
6
5
4
3
2
1
0
Stack in mystack2:
19
18
17
16
15
14
13
12
11
10

```

A Closer Look at Methods and Classes:

Overloading methods

In Java it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different. When this is the case, the methods are said to be *overloaded*, and the process is referred to as *method overloading*. Method overloading is one of the ways that Java supports polymorphism. When an overloaded method is invoked, Java uses the type and/or number of arguments as its guide to determine which version of the overloaded method to actually call. Thus, overloaded methods must differ in the type and/or number of their parameters. While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method. When Java encounters a call to an overloaded method, it simply executes the version of the method whose parameters match the arguments used in the call.

Example Program

```

class OverloadDemo {
void test() {
System.out.println("No parameters");
}
}

```

```

}
// Overload test for one integer parameter.
void test(int a) {
    System.out.println("a: " + a);
}

// Overload test for two integer parameters.
void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
}

// overload test for a double parameter
double test(double a) {
    System.out.println("double a: " + a);
    return a*a;
}

}
class Overload {
public static void main(String args[]) {
    OverloadDemo ob = new OverloadDemo();
    double result;
    // call all versions of test()
    ob.test();
    ob.test(10);
    ob.test(10, 20);
    result = ob.test(123.25);
    System.out.println("Result of ob.test(123.25): " + result);
}
}

```

Output

```

No parameters
a: 10
a and b: 10 20
double a: 123.25
Result of ob.test(123.25): 15190.5625

```

Method **test()** is overloaded four times. The first version takes no parameters, the second takes one integer parameter, the third takes two integer parameters, and the fourth takes one **double** parameter. The fourth version of **test()** also returns a value is of no consequence relative to overloading, since return types do not play a role in overload resolution.

Java looks for a match between the arguments used to call the method and the method's parameters. However, this match need not always be exact. In some cases, Java's automatic type conversions can play a role in overload resolution.

```

// Automatic type conversions apply to overloading.
class OverloadDemo {
void test() {
    System.out.println("No parameters");
}

// Overload test for two integer parameters.
void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
}

// overload test for a double parameter
void test(double a) {

```

```

System.out.println("Inside test(double) a: " + a);
    }
}
class Overload {
public static void main(String args[]) {
OverloadDemo ob = new OverloadDemo();
int i = 88;
ob.test();
ob.test(10, 20);
ob.test(i); // this will invoke test(double)
ob.test(123.2); // this will invoke test(double)
    }
}

```

Output

```

No parameters
a and b: 10 20
Inside test(double) a: 88
Inside test(double) a: 123.2

```

This version of **OverloadDemo** does not define **test(int)**. Therefore, when **test()** is called with an integer argument inside **Overload**, no matching method is found. However, Java can automatically convert an integer into a **double**, and this conversion can be used to resolve the call. Therefore, after **test(int)** is not found, Java elevates **i** to **double** and then calls **test(double)**. Of course, if **test(int)** had been defined, it would have been called instead. Java will employ its automatic type conversions only if no exact match is found.

Overloading Constructors

In addition to overloading normal methods, constructor methods can also be overloaded. For most real-world classes that are created, overloaded constructors will be the norm.

```

class Box {
    double width;
    double height;
    double depth;

    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    double volume() {
        return width * height * depth;
    }
}

```

Box() constructor requires three parameters. All declarations of **Box** objects must pass three arguments to the **Box()** constructor. The following statement is currently invalid:

```
Box ob = new Box();
```

Since **Box()** requires three arguments, it's an error to call it without them. As a solution to this the **Box** constructor can be overloaded so that it handles the above situation.

```
class Box {
double width;
double height;
double depth;
// constructor used when all dimensions specified
Box(double w, double h, double d) {
width = w;
height = h;
depth = d;
}
// constructor used when no dimensions specified
Box() {
width = -1; // use -1 to indicate
height = -1; // an uninitialized
depth = -1; // box
}
// constructor used when cube is created
Box(double len) {
width = height = depth = len;
}
// compute and return volume
double volume() {
return width * height * depth;
}
}
class OverloadCons {
public static void main(String args[]) {
// create boxes using the various constructors
Box mybox1 = new Box(10, 20, 15);
Box mybox2 = new Box();
Box mycube = new Box(7);
double vol;

// get volume of first box
vol = mybox1.volume();
System.out.println("Volume of mybox1 is " + vol);
// get volume of second box
vol = mybox2.volume();
System.out.println("Volume of mybox2 is " + vol);
// get volume of cube
vol = mycube.volume();
System.out.println("Volume of mycube is " + vol);
}
}
```

Output

```
Volume of mybox1 is 3000.0
Volume of mybox2 is -1.0
Volume of mycube is 343.0
```

Using Objects as Parameters

Objects can be passed as parameters into methods or constructors.

Example Program.

```
class Test {
    int a, b;
    Test(int i, int j) {
        a = i;
        b = j;
    }
    // return true if o is equal to the invoking object
    boolean equals(Test o) {
        if(o.a == a && o.b == b) return true;
        else return false;
    }
}

class PassOb {
    public static void main(String args[]) {
        Test ob1 = new Test(100, 22);
        Test ob2 = new Test(100, 22);
        Test ob3 = new Test(-1, -1);
        System.out.println("ob1 == ob2: " + ob1.equals(ob2));

        System.out.println("ob1 == ob3: " + ob1.equals(ob3));
    }
}
```

Output

```
ob1 == ob2: true
ob1 == ob3: false
```

The **equals()** method inside **Test** compares two objects for equality and returns the result. It compares the invoking object with the one that it is passed. If they contain the same values, then the method returns **true**. Otherwise, it returns **false**. The parameter **o** in **equals()** specifies **Test** as its type.

In order to construct a new object that it is initially the same as some existing object, a constructor that takes an object of its class as a parameter has to be defined. The example below shows how an object can be passed as a parameter to a constructor.

```
class Box {
    double width;
    double height;
    double depth;
    // Notice this constructor. It takes an object of type Box.
    Box(Box ob) { // pass object to constructor
        width = ob.width;
        height = ob.height;
        depth = ob.depth;
    }
    // constructor used when all dimensions specified
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }
}
```

```

}
// constructor used when no dimensions specified
Box() {
width = -1; // use -1 to indicate
height = -1; // an uninitialized
depth = -1; // box
}
// constructor used when cube is created
Box(double len) {
width = height = depth = len;
}
double volume() {
return width * height * depth;
}
}
class OverloadCons2 {
public static void main(String args[]) {
// create boxes using the various constructors
Box mybox1 = new Box(10, 20, 15);
Box mybox2 = new Box();
Box mycube = new Box(7);
Box myclone = new Box(mybox1); // create copy of mybox1
double vol;
// get volume of first box
vol = mybox1.volume();
System.out.println("Volume of mybox1 is " + vol);
// get volume of second box
vol = mybox2.volume();
System.out.println("Volume of mybox2 is " + vol);
// get volume of cube
vol = mycube.volume();
System.out.println("Volume of cube is " + vol);
// get volume of clone
vol = myclone.volume();
System.out.println("Volume of clone is " + vol);
}
}

```

Argument Passing

There are two ways in which an argument can be passed to a subroutine.

1. Call by value
2. Call by reference

The Call by value approach, copies the *value* of an argument into the formal parameter of the subroutine. Therefore, changes made to the parameter of the subroutine have no effect on the argument. In the second approach (call by reference), a reference to an argument is passed to the parameter. Inside the subroutine, this reference is used to access the actual argument specified in the call. The changes made to the parameter will affect the argument used to call the subroutine.

Program that demonstrates pass by value

```

class Test {
void meth(int i, int j) {
i *= 2;
j /= 2;
}
}

```

```

    }
}
class CallByValue {
public static void main(String args[]) {
Test ob = new Test();
int a = 15, b = 20;
System.out.println("a and b before call: " + a + " " + b);
ob.meth(a, b);
System.out.println("a and b after call: " + a + " " + b);
}
}

```

Output

```

a and b before call: 15 20
a and b after call: 15 20

```

The operations that occur inside **meth()** have no effect on the values of **a** and **b** used in the call; their values here did not change to 30 and 10.

When an object is passed to a method, it is a call by reference variant. When a reference is passed to a method, the parameter that receives it will refer to the same object as that referred to by the argument. Changes to the object inside the method *do* affect the object used as an argument.

Program that demonstrate pass by reference

```

class Test {
int a, b;
Test(int i, int j) {
a = i;
b = j;
}
void meth(Test o) {
o.a *= 2;
o.b /= 2;
}
}
class CallByRef {
public static void main(String args[]) {
Test ob = new Test(15, 20);
System.out.println("ob.a and ob.b before call: " + ob.a + " " + ob.b);
ob.meth(ob);
System.out.println("ob.a and ob.b after call: " + ob.a + " " + ob.b);
}
}

```

Output

```

ob.a and ob.b before call: 15 20
ob.a and ob.b after call: 30 10

```

The actions inside **meth()** have affected the object used as an argument.

Returning objects

A method can return any type of data, including the class types that are created.

Program to demonstrate how objects are returned.

```

class Test {
    int a;
    Test(int i) {
        a = i;
    }
    Test incrByTen() {
        Test temp = new Test(a+10);
        return temp;
    }
}
class RetOb {
    public static void main(String args[]) {
        Test ob1 = new Test(2);
        Test ob2;
        ob2 = ob1.incrByTen();
        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);
        ob2 = ob2.incrByTen();
        System.out.println("ob2.a after second increase: "+ ob2.a);
    }
}

```

Output

```

ob1.a: 2
ob2.a: 12
ob2.a after second increase: 22

```

Each time **incrByTen()** is invoked, a new object is created, and a reference to it is returned to the calling routine.

Access Control

Encapsulation links data with the code that manipulates it. Encapsulation provides another important attribute: *access control*. Java's access specifiers are **public**, **private**, and **protected**. Java also defines a default access level. **protected** applies only when inheritance is involved. When a member of a class is modified by the **public** specifier, then that member can be accessed by any other code. When a member of a class is specified as **private**, then that member can only be accessed by other members of its class. **main()** is always preceded by the **public** specifier. It is called by code that is outside the program—that is, by the Java run-time system. When no access specifier is used, then by default the member of a class is public within its own package, but cannot be accessed outside of its package. An access specifier precedes the rest of a member's type specification. Here is an example:

```

public int i;
private double j;
private int myMethod(int a, char b)

```

Program that demonstrates difference between public and private.

```

class Test {
    int a; // default access
    public int b; // public access
    private int c; // private access
    // methods to access c
}

```



```

void setc(int i) { // set c's value
c = i;
}
int getc() { // get c's value
return c;
}
}
class AccessTest {
public static void main(String args[]) {
Test ob = new Test();
// These are OK, a and b may be accessed directly
ob.a = 10;
ob.b = 20;
// This is not OK and will cause an error
// ob.c = 100; // Error!
// You must access c through its methods
ob.setc(100); // OK
System.out.println("a, b, and c: " + ob.a + " " +
ob.b + " " + ob.getc());
}
}

```

Inside the **Test** class, **a** uses default access, which for this example is the same as specifying **public**. **b** is explicitly specified as **public**. Member **c** is given private access. It cannot be accessed by code outside of its class. Inside the **AccessTest** class, **c** cannot be used directly. It must be accessed through its public methods: **setc()** and **getc()**.

The Static keyword

When there is a need to define a class member that will be used independently of any object of that class, the **static** keyword should be used. It is possible to create a member that can be used by itself, without reference to a specific instance. To create such a member, precede the member's declaration with the keyword **static**. When a member is declared **static**, it can be accessed before any objects of its class are created, and without reference to any object. Both methods and variables can be defined to be **static**.

Instance variables declared as **static** are, essentially, global variables. When objects of its class are declared, no copy of a **static** variable is made. Instead, all instances of the class share the same **static** variable.

Methods declared as **static** have several restrictions:

- They can only call other **static** methods.
- They must only access **static** data.

If any computation needs to be done in order to initialize static variables, a static block can be declared. This block will get executed only once, when the class is first loaded.

Program that demonstrates static variables, methods, and blocks.

```

class UseStatic {
static int a = 3;
static int b;
static void meth(int x) {
System.out.println("x = " + x);
System.out.println("a = " + a);
}
}

```

```

System.out.println("b = " + b);
}
static {
System.out.println("Static block initialized.");
b = a * 4;
}
public static void main(String args[]) {
meth(42);
}
}

```

As soon as the **UseStatic** class is loaded, all of the **static** statements are run. First, **a** is set to **3**, then the **static** block executes, which prints a message and then initializes **b** to **a * 4** or **12**. Then **main()** is called, which calls **meth()**, passing **42** to **x**. The three **println()** statements refer to the two **static** variables **a** and **b**, as well as to the local variable **x**.

Output

```

Static block initialized.
x = 42
a = 3
b = 12

```

Outside of the class in which they are defined, **static** methods and variables can be used independently of any object. To do so, one needs to only specify the name of the class followed by the dot operator.

Program to demonstrate how static variables and methods can be accessed.

```

class StaticDemo {
static int a = 42;
static int b = 99;
static void callme() {
System.out.println("a = " + a);
}
}
class StaticByName {
public static void main(String args[]) {
StaticDemo.callme();
System.out.println("b = " + StaticDemo.b);
}
}

```

Output

```

a = 42
b = 99

```

Final keyword

A variable can be declared as **final**. Doing so prevents its contents from being modified. A **final** variable must be initialized when it is declared.

```

final int FILE_NEW = 1;
final int FILE_OPEN = 2;

```

It is a common coding convention to choose all uppercase identifiers for **final** variables.

Command Line Arguments

If the user wishes to pass information into a program when he runs it, it can be done through command line arguments to `main()`. A command-line argument is the information that directly follows the program's name on the command line when it is executed. The command line arguments are stored as strings in the **String** array passed to the `args` parameter of `main()`. The first command-line argument is stored at **`args[0]`**, the second at **`args[1]`**, and so on.

Program that displays command line arguments.

```
class CommandLine {
public static void main(String args[]) {
for(int i=0; i<args.length; i++)
System.out.println("args[" + i + "]: " +
args[i]);
}
}
```